HEART: Coordination of Heterogeneous Expert Agents for Physically Grounded Robotic Task Planning

Anonymous Submission

***Abstract*— Large Language Models (LLMs) can reason over complex instructions but often fail to satisfy the physical and spatial constraints required for robotic task planning. Recent LLM-based planners directly translate text into action sequences, yet they lack structured reasoning about feasibility, reachability, and logical order, resulting in invalid or incomplete plans. We present a heterogeneous multi-LLM framework that decomposes instructions into atomic reasoning tasks and allocates them to role-specialized expert agents under a token budget for real-world communication and computation constraints. By combining role-oriented reasoning from heterogeneous agents followed by constraint-driven plan synthesis, HEART validates capability, reachability, and constraint conditions before planning and produces physically executable plans while maintaining efficiency. Experiments across three household benchmarks show that HEART consistently improves plan success and token efficiency compared to single-LLM and rule-based planners, demonstrating that heterogeneous LLM collaboration enables robust and scalable robotic task planning under resource constraints.**

I. Introduction

Large Language Models (LLMs) have recently gained attention in robotic task planning [1]–[4] as they can deal with natural language and reason without explicit symbolic domain models. This flexibility contrasts with classical planners [5], [6], which rely on predefined operators and often fail under ambiguous instructions. However, robotic task planning in physical environments must satisfy two conditions: logical validity and physical executability. Logical validity ensures consistent action order and preconditions, while physical executability ensures robot capabilities, reachability, and feasibility. Single LLMs often struggle to handle these requirements jointly, generating plans that appear reasonable in language but fail when executed on real robots. For example, when asked to "serve food to the dining table by putting any single apple," a single LLM may create a plausible plan (e.g., $navigate$, $pick$, $navigate$, $place$) but often does not consider whether the robot is holding the apple before executing the $place$ action (i.e., logically invalid) or whether the robot can grasp the apple with its mechanical design (i.e., physically inexecutable).

To generate task plans that are both valid and executable, reasoning should jointly consider capability, environment, path, feasibility, and constraint. However, those methods based on a single LLM struggle to integrate these aspects, as they must process large and often irrelevant context [7], making prompts unnecessarily long to account for the physical properties of robots and their environments. A naïve multi-LLM approach, where agents reason in parallel or sequence without coordination, still fails to produce consistent reasoning [8]. Each agent receives the same full context, leading to redundant processing, excessive token usage, and conflicting reasoning results that compromise plan validity and executability [9]–[11]. Therefore, LLM-based robotic task planning faces two key challenges: achieving valid and executable plans through heterogeneous reasoning, and maintaining efficiency in multi-agent collaboration by allocating reasoning tasks to the most relevant agents under communication constraints [12]–[14].

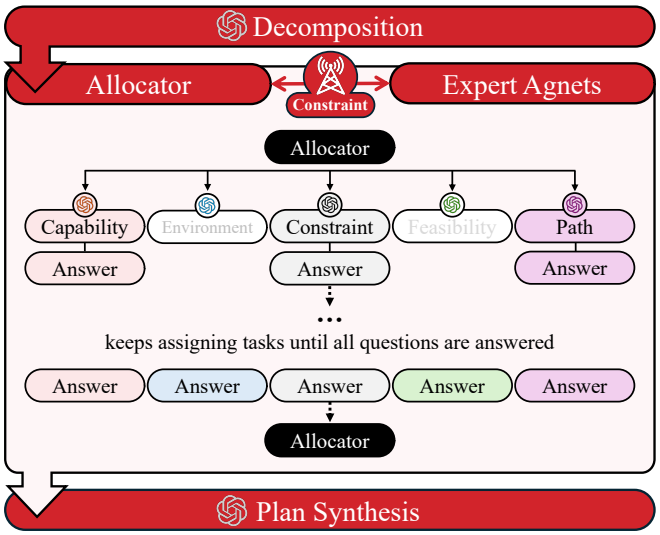

Fig. 1: An overview of HEART. The framework decomposes the planning process into reasoning questions, assigns them to role-specialized expert agents through the allocator, and synthesizes their responses to generate a valid and physically executable plan.

We propose Heterogeneous Expert Agents for physically grounded Robotic Task planning (HEART), a coordinated multi-LLM framework that combines heterogeneous reasoning with communication constraint allocation for robotic task planning. As illustrated in Fig. 1, HEART decomposes the planning problem into atomic reasoning questions and assigns them to five role-specialized agents (i.e., *capability*, *environment*, *path*, *feasibility*, and *constraint*) under a shared token budget. A central allocator coordinates this process by assigning each question to the most relevant agent and distributing token budgets across roles to limit redundancy and maintain efficiency under communication constraints. Each agent focuses only on information relevant to its role, thereby improving reasoning precision, and their outputs are integrated in the planning stage to produce valid and physically executable task plans. This role-based structure jointly

satisfies logical and physical constraints, while coordinated allocation prevent redundant computation and token usage within communication limits. To the best of our knowledge, HEART is the first multi-LLM framework that explicitly integrates heterogeneous reasoning and communication-efficient coordination to achieve physically grounded and executable robotic task planning.

II. Related Work

LLMs have been explored as planners that map natural language instructions to executable action sequences. Early approaches such as SayCan [15], PaLM-E [3], and SayPlan [1] grounded instructions in affordances, multimodal perception, or scene graphs to generate plans linked to the real environment. Subsequent studies, including SMART-LLM [16], extended this idea to multi-robot settings but still faced limited precision and reasoning depth [17], [18]. These limitations motivated LLM-as-translator frameworks that convert planning problems into formal representations such as PDDL [2], [19] or temporal logics [20], [21]. These LLM-as-translator approaches improve logical consistency from symbolic solvers and validation tools. However, because the translation step separates symbolic reasoning from physical and spatial context, these methods still struggle to capture diverse reasoning aspects such as spatial coordination, physical feasibility, and temporal constraints, often resulting in failed execution [21]–[23].

Recent work has also explored multi-agent LLM frameworks for robotic task planning, where multiple agents collaborate to extend reasoning consistency in planning. RoCo [8] assigns one agent per robot and coordinates task and motion planning through dialogue and feedback. Other frameworks [24], [25] focus on collaborative problem formulation and task decomposition, where multiple LLMs assume complementary roles to jointly construct domain models and refine planning steps. Multi-agent LLM systems have also been proposed for policy generation and control synthesis, such as MAS for robotic autonomy [26] and Triple-S [27] for long-horizon policy design. A related approach, FlowPlan [28] uses LLMs with multi-step logical flows to capture operational constraints in task instructions. Despite these advances, most existing multi-agent LLM frameworks still lack explicit mechanisms for coordinating reasoning under communication constraints [12]. Without efficient task allocation, agents often duplicate reasoning over overlapping context, reach inconsistent results, and fail to scale when token budgets are limited. These limitations show the need for a systematic allocation strategy that distributes reasoning questions efficiently and maintains consistency across agents operating within communication limits [10], [11], [14].

Beyond LLM-based methods, resource allocation has long been a central issue in multi-robot and distributed systems, from classical task assignment and market-based optimization [29]–[31] to adaptive scheduling in heterogeneous clusters [32]. Recent compute-allocation studies in LLM inference [10], [13], [33] further show that careful budgeting of reasoning resources enhances robustness and efficiency. These insights motivate our token-aware allocation strategy for multi-agent reasoning.

III. Problem Statement

We consider the problem of LLM-based, instruction-driven robot task planning, where natural language instructions should be mapped to task plans that are both valid and executable. The input consists of high-level instruction I and environment data $E = (R, G)$ where R includes robot specifications and current states and G is a 3D scene graph [34] describing the environment with rooms, connectivity, and objects with affordances and physical properties. The output is a plan π, expressed as a sequence of symbolic task actions, that is *valid* (i.e., satisfying logical and temporal constraints of the task) and *executable* (i.e., respecting robot capabilities and physical feasibility).

The challenge is twofold: (i) plans must consider robot capability, physical and spatial feasibility, and logical order of actions to ensure the validity and executability; (ii) reasoning must consider limited computational and memory resources, since expanding the context size increases computational cost, memory usage, and inference latency. Thus, we introduce a per-round token budget b that restricts the amount of communicated tokens per each reasoning. Our objective is to produce valid and executable task plans while maintaining efficiency under the token budget.

While this problem applies broadly across robotic application domains, we focus on household mobile manipulation tasks as a running example throughout the paper. In this example, robots must navigate and manipulate objects (e.g., pick up, open, turn on) in structured indoor environments, which are general capabilities across varied domains.

IV. Methods

We develop HEART, a token-aware multi-LLM framework for robotic task planning that coordinates heterogeneous expert agents through a central allocator. The HEART pipeline consists of four stages, illustrated in Fig. 2. (i) **Task decomposition** translates the instruction into atomic reasoning questions, which are derived from the environment data by identifying what additional information is required to accomplish the task. We regard these questions as subtasks, each resolved through its corresponding answer. (ii) **Task allocation** assigns these subtasks to expert agents under a per-round token budget b. (iii) **Heterogeneous expert reasoning** executes the assigned subtasks in parallel, with each agent operating on role-aligned and filtered data. Stages (ii)—(iii) repeat until all subtasks are resolved or a termination condition is met. (iv) **Plan synthesis** integrates the resulting Q&A outcomes to generates valid and executable task plans.

A. Task Decomposition

The first stage of HEART decomposes a high-level instruction I into atomic reasoning subtasks $\mathcal{Q} = \{q_i\}_{i=1}^{N}$. These subtasks represent distinct reasoning requirements in robot task planning, including capability checking, physical feasibility assessment, environment understanding, path finding,

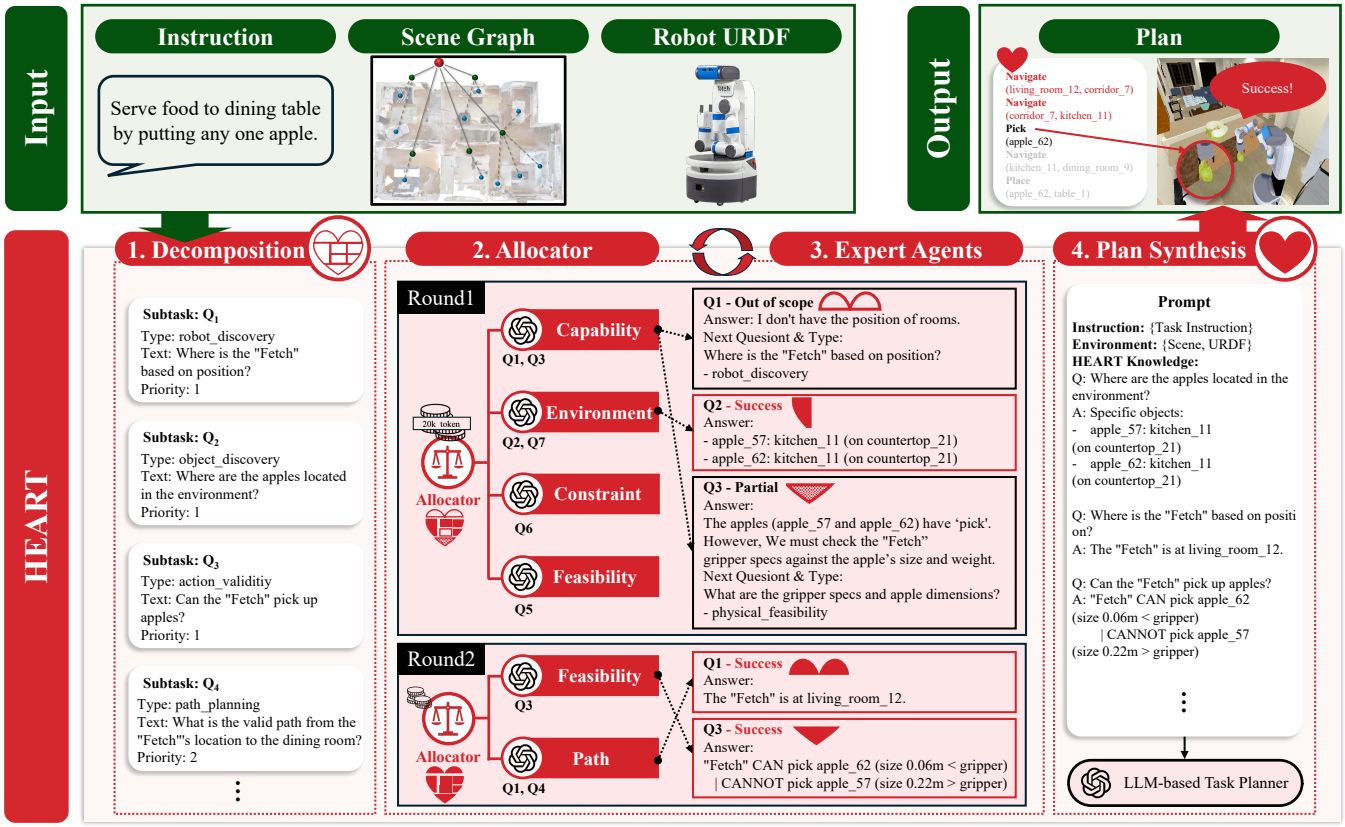

Fig. 2: Overview of the HEART pipeline. Natural language instructions, scene graphs, and robot URDFs are decomposed into subtasks, allocated to heterogeneous expert agents under a token budget, and resolved through role-aligned reasoning. Agent responses (e.g., object discovery, feasibility checks) are integrated into planner inputs, producing executable task plans. The example shows a Fetch robot performing *"Serve food to the dining table by putting any one apple"*, decomposed into subtasks (Q1–Q7) and synthesized into a valid, executable plan.

TABLE I: Summary of reasoning task types. Each type defines a specific reasoning focus required for generating valid and executable plans. The rightmost column shows the fixed agent mapping used only by the Type-Based baseline allocator (Sec. V-C).

Task Type	Brief Description	Typebased Agent
Robot Capability	Identify Robot Capability	Capability
Action Validity	Check Action-Object Match	Capability
Object Discovery	Find Objects in Scene	Environment
Scene Understanding	Analyze Spatial Relationship	Environment
Robot Discovery	Determine Robot Location	Path
Path Planning	Find Valid Navigation Path	Path
Route Optimization	Select Most Efficient Path	Path
Physical Feasibility	Calculate Robot Physical	Feasibility
Task Dependency	Extract Action Prerequisites	Constraint
Action Constraint	Derive Partial Order Plan	Constraint
Contextual Constraint	Identify Implicit Rules	Constraint

and constraint extraction. Each subtask is labeled with one of the 11 task types defined in Table I, which specify its reasoning focus and determine the expert agent responsible for processing it. These task types capture the commonly required reasoning needs across recent LLM-based robotic planning works [3], [4], [8], [15], [16], [20], [26].

A subtask is defined as $q_i = (x_i, r_i, \tau_i, p_i, stat_i)$, where x_i is the reasoning question, r_i the answer, $\tau_i \in \mathcal{T}$ the task type, p_i a local priority, and a status flag $stat_i \in \{\texttt{SUCCESS}, \texttt{PARTIAL}, \texttt{OUT_OF_SCOPE}\}$. Subtasks are initialized with $r_i = \varnothing$ and $stat_i = \texttt{PENDING}$. The decomposer agent, implemented as an LLM, derives x_i as the reasoning question required to complete the instruction I, selects a suitable task type τ_i, and assigns a priority p_i that reflects the importance of resolving this subtask among subtasks, thereby instantiating q_i in the above format. During expert reasoning (Sec. IV-C), subtasks are updated: r_i is populated, x_i or τ_i may be updated to coordinate with other agents, and $stat_i$ is set to SUCCESS, PARTIAL, or OUT_OF_SCOPE.

In our example shown in Fig. 2, the instruction "Serve food to the dining table …" requires information related to the robot navigation and manipulation. For navigation, HEART must identify the current location of the robot and its goals, such as the positions of the food (e.g., apples) and the dining table. It must also determine whether the robot is capable of picking and placing the food. These pieces of information correspond to subtasks categorized as robot discovery, object discovery, and action validity, respectively (corresponding to Q1, Q2, Q3 in Fig. 2).

B. Allocation under Token Budget

The allocator centrally maps subtasks from $\mathcal{Q}$ to expert agents $\mathcal{A}$ under a per-round token budget b, which models limited computational and memory resources.[1] Each LLM

[1]We assume that the value of b is given as calculating an exact value for a specific system is out of scope.

Algorithm 1 ALLOCATOR

1: **Input:** Subtasks $\mathcal{Q}$, expert agents $\mathcal{A}$, budget b
2: **Output:** Assignment set $M = \{(q_i, \varphi(q_i))\}$
3: **for** each $q_i \in \mathcal{Q}$ **do**
4: Compute $s_i(a)$ with embeddings and history penalty
5: $\varphi(q_i) \leftarrow \arg\max_{a \in \mathcal{A}} s_i(a)$ ..... // Sec. IV-B.1
6: **end for**
7: Compute demand for each a from $\{\varphi(q_i) \mid q_i \in \mathcal{Q}\}$
8: Plan capacities y_a s.t. $\sum_{a \in \mathcal{A}} y_a\, c_a \leq b$. // Sec. IV-B.2
9: **for** each $a \in \mathcal{A}$ **do**
10: Select up to y_a subtasks for a by priority p_i
11: **end for**
12: $M \leftarrow \{(q_i, \varphi(q_i)) \mid q_i$ dispatched in this round$\}$
13: Update allocation history
14: **return** M

agent $a \in \mathcal{A}$ is represented by $a = (d_a, F(a, E), c_a)$, where d_a is its role description, $F(a, E)$ is the role-aligned filtered environment data, and c_a is an estimated token usage based on prompt size and past query history. This representation allows the allocator to match subtasks to suitable agents and approximate token consumption during scheduling.

This stage, described in Alg. 1, addresses two key requirements: (i) Subtask assignment, which combines semantic relevance with history-sensitive rescheduling of unresolved subtasks; and (ii) Capacity planning, which ensures the token budget while maintaining balanced utilization across agents.

1) Subtask assignments with history-sensitive penalty: For each subtask q_i with reasoning question x_i and task type τ_i, the allocator computes a semantic score over agents using text embeddings from Sentence-BERT [35]:

$$s_i(a) \;=\; \cos\big(e(x_i), e(d_a)\big) \;+\; \cos\big(e(\tau_i), e(d_a)\big) \;-\; \Delta_i(a),$$

where d_a is the role description of the agent a, $e(\cdot)$ denotes the embedding function, and $\Delta_i(a)$ is a history-sensitive penalty that increases with previous failed attempts on a and with the most recent failed agent. Without this penalty, a subtask may be repeatedly assigned to agents that have already failed. The selected agent is $\varphi(q_i) = \arg\max_{a \in \mathcal{A}} s_i(a)$. To avoid unbounded retries, each subtask is assigned a maximum trial limit (set to five in our experiments). Subtasks that exceed this limit without achieving $stat_i = \texttt{SUCCESS}$ are marked as $\texttt{FAIL}$ and excluded from further allocations.

2) Capacity planning with token budget: From the subtask assignment, we compute the demand for each agent type based on the remaining unresolved subtasks. Using estimated token count c_a, the allocator computes capacities y_a under the budget b (in tokens):

$$\sum_{a \in \mathcal{A}} y_a\, c_a \;\leq b, \qquad y_a \in \mathbb{Z}_{\geq 0}.$$

We adopt a greedy allocation strategy that iterates over agents in decreasing order of token count c_a, assigning as many instances as possible until the demand or budget is exhausted. If $c_a > b$, only one instance of a is allowed. Subtasks are then scheduled by priority p_i, and the allocator outputs an assignment set $M = \{(q_i, \varphi(q_i))\}$ for the round. The estimated token count is updated after each round to refine future capacity planning. Failed attempts are also recorded for each subtask (line 13).

In the example in Fig. 2, subtasks such as Q1 (robot discovery), Q2 (object discovery), and Q3 (action validity) are initially assigned based on semantic similarity. Due to the token budget, only a subset is dispatched in the first round, and the rest are rescheduled by priority in subsequent rounds. Subtasks that fail or return partial answers (e.g., Q1 and Q3) are updated and reassigned in later rounds, allowing HEART to progressively refine incomplete reasoning.

C. Heterogeneous Expert Reasoning

The third phase executes allocated subtasks using heterogeneous expert agents. Each agent specializes in a distinct reasoning aspect required for robotic task planning. To support role specialization, each agent receives only the role-aligned filtered data $F(a, E)$ introduced earlier, rather than the full scene. This reduces context size and redundancy, improves efficiency and reasoning quality, and aligns with reports that modular, specialized models are more economical and robust than monolithic ones [36].

Given an assigned subtask $q_i = (x_i, r_i, \tau_i, p_i, stat_i)$, agent a processes it using the reasoning question x_i together with $F(a, E)$ and returns an updated answer r_i with status flag $stat_i \in \{\texttt{SUCCESS}, \texttt{PARTIAL}, \texttt{OUT_OF_SCOPE}\}$. Agents operate in parallel within each round as determined by the allocator; some may process multiple subtasks while others remain idle. The five instantiated roles are:

Capability Reasoner evaluates what the robot can do in its current configuration. Using URDF and robot state (kinematics, joint limits, base mobility, gripper type, predefined action set), it determines whether specified navigation or manipulation actions are supported for given objects and what interaction modes are allowed.

Environment Reasoner focuses on scene and object understanding. From the information provided by 3D scene graph (e.g., rooms, objects, affordances, accessibility flags, container relations), it identifies relevant objects, their states, and spatial context needed by subsequent reasoning.

Path Reasoner plans high-level navigation sequences based on the connectivity between spaces. Using room adjacency, spatial coordinates, and the current robot pose, it decides whether the required locations are mutually reachable and returns room-sequence routes that respect adjacency.

Feasibility Reasoner validates the physical feasibility of candidate actions. It combines the kinematic information of the robot (e.g., arm length, gripper size) of the robot with object properties (e.g., dimensions, weight, material) to judge graspability, reachability, and payload limits for the action.

Constraint Reasoner extracts logical and temporal constraints from the instruction and environment. Using affordances, object states, and accessibility information, it derives ordering relations (e.g., open the gripper, pick an object, and close the gripper, resulting in an action sequence

$open, pick, close$) and resolves ambiguities to produce action dependencies for a consistent plan.

All expert agents follow the same prompting schema to standardize inputs and outputs across roles. The general template is shown below, where role-level fields (fixed per agent) are highlighted in red, and subtask-specific fields (varying per subtask) are highlighted in green.

System

You are a **Specific-Agent** specialized in robotic task planning. Your primary role is to **Specific-Agent Purpose**, supported by a role-aligned view of the environment that provides **Specific-Agent-Aligned Data Description**. In this role, you reason about **Specific-Agent Reasoning**, following **Specific-Agent Rules** that constrain your scope. Through this analysis you contribute **Specific-Agent Contribution**, which is integrated into the broader planning process.

Human

Task
Question: **Question**
Current Focus: **Question Type**
Original Goal: **Original Question**
Original Type: **Original Question Type**

Agent Data
Your Data: **Specific-Agent-Aligned Data**
Previous Context: **Specific-Agent Memory Data**

Response Requirements
Current Response Format: **Current Task Response Format**
Original Response Format: **Original Task Response Format**

Decision Guidelines
Can you answer the ORIGINAL question in the required format? → `SUCCESS`
Can you contribute ANY useful information? → `PARTIAL`
Is it completely outside your expertise with no relevant data? → `OUT_OF_SCOPE`

Response Format
Status: `SUCCESS` / `PARTIAL` / `OUT_OF_SCOPE`
Answer: Based only on verified data; include any details helpful to the next agent.
Follow-up Question: (if not `SUCCESS`) Minimal extra info needed, phrased for the appropriate agent.
Follow-up Task Type: (if not `SUCCESS`) Choose from available task types.

Each response of the agents updates r_i and sets $stat_i$ according to the completion criteria of the current type τ_i: `SUCCESS` when all required fields are present, `PARTIAL` when only a subset is available, and `OUT_OF_SCOPE` when no role-relevant inference is possible. For `PARTIAL` or `OUT_OF_SCOPE`, the agent rephrases x_i and retags τ_i to propose a minimal follow-up for later rounds.

Completion of response is type-specific: the prompt schema enumerates the required fields for each τ_i (e.g., *capability* requires an `action_set`, *path* requires `valid_paths`, and *feasibility* requires boolean checks of specified physical properties). Each agent maintains a local memory of its prompts and responses. This memory can be re-injected in later rounds to preserve consistency and improve robustness when handling partial answers or failures.

In Fig. 2, Q1 (robot discovery) is first handled by the Capability Reasoner, which returns `OUT_OF_SCOPE` because location inference lies outside its role. Q3 (action validity) also initially goes to the Capability Reasoner, but only a `PARTIAL` answer is produced because determining whether the robot can grasp an apple requires additional information (e.g., object dimensions) from other agents. These unresolved subtasks are returned to the allocator for reassignment. In contrast, Q2 (object discovery) is directly resolved by the Environment Reasoner, which has access to object information, allowing the subtask to be marked as `SUCCESS`.

D. Plan Synthesis

The final phase of HEART synthesizes an executable task plan π from the resolved subtasks $\mathcal{Q}$. Each subtask $q_i = (x_i, r_i, \tau_i, p_i, stat_i)$ contributes facts or constraints, and the collection of (x_i, r_i, τ_i) forms the planning context. The synthesis stage augments the input to the planner by constructing a planner-specific prompt or problem specification with this context, so that capability limits, feasibility results, discovered objects, and ordering relations are enforced during plan generation. Following the general Q&A reasoning paradigm explored in LLM workflows [37], HEART represents the obtained information as question-answer pairs and integrates them into the planner prompts. HEART can work with any LLM-based or hybrid planner that accepts natural language or structured textual input. The output is a plan π that satisfies logical constraints and physical feasibility with respect to the robot and the environment. If no plan is possible, the system returns an explicit infeasibility report.

In Fig. 2, the resolved subtasks determine the planning context: Q1 provides the initial location of the robot, Q2 identifies all apples and their states, and Q3 determines which apples are physically graspable. These Q&A outputs are merged into the planner input, selecting a feasible target apple, specifying reachable locations, and enforcing the ordering constraints to generate a valid and executable plan.

V. Experiments

We design three experiments to evaluate HEART across different aspects of reasoning and performance: (i) **heterogeneous vs. homogeneous agents**, testing whether role specialization improves planning robustness and efficiency; (ii) **allocator efficiency**, comparing HEART allocator against alternative strategies under varying budgets; and (iii) **task planning performance**, evaluating whether HEART improves the validity and executability of plans across both LLM-based and symbolic planners.

A. Experimental Setup

1) Dataset: We evaluate HEART on three household manipulation scenes from Gibson dataset [38], each chosen to emphasize different reasoning challenges. **Beechwood** (Fig. 3a) contains 9 rooms and 74 objects (15 affordances) and emphasizes logical sequencing. For example, tasks such

TABLE II: Comparison of heterogeneous vs. homogeneous agents. Results are averaged over 50 random seeds (mean ± 95% CI). Metrics include subtask size (#Subtasks), allocation efficiency (Rounds, Avg. Tokens / Subtask, Avg. Time / Subtask, Subtask SR), and planning quality (Plan SR).

Scene	Agent	#subtasks	Rounds	Avg. Time/subtask (s)	Avg. tokens/subtask (k)	Subtask SR (%)	Plan SR (%)
Beechwood	Homo	11.9 ± 0.7	10.9 ± 0.7	7.5 ± 0.3	12.7 ± 0.0	97.5	76.0
	Hetero	11.9 ± 1.0	5.4 ± 0.4	5.4 ± 0.7	8.8 ± 0.6	95.0	82.0
Benevolence	Homo	11.7 ± 0.5	6.0 ± 0.5	4.0 ± 0.2	10.1 ± 0.0	99.3	50.0
	Hetero	11.9 ± 0.5	4.8 ± 0.5	4.2 ± 0.4	7.5 ± 0.5	98.7	74.0
Merom	Homo	15.1 ± 1.0	15.1 ± 1.0	10.6 ± 1.0	12.0 ± 0.0	99.5	42.0
	Hetero	15.2 ± 0.9	6.7 ± 0.5	6.3 ± 0.5	8.8 ± 0.5	96.0	60.0

Fig. 3: Household manipulation scenes used for evaluation: (a) **Beechwood**, featuring 9 rooms and 74 objects; (b) **Benevolence**, with 5 rooms and 55 objects; (c) **Merom**, with 7 rooms and 65 objects. Figures are adapted from the 3D Scene Graph dataset [34].

as laundry (requiring opening the washer before loading clothes) and kitchen safety (e.g., turning off an active stove) involve clear preconditions and ordered action dependencies. **Benevolence** (Fig. 3b) consists of 5 rooms and 55 objects (11 affordances). It stresses physical feasibility: oversized apples that cannot be grasped, heavy sunglasses exceeding robot payload, and objects located at unreachable heights require URDF-based reasoning over robot kinematic constraints and payloads. **Merom** (Fig. 3c) includes 7 rooms and 65 objects (16 affordances). It evaluates the performance on multi-robot cooperation with both heterogeneous teams (a Fetch mobile manipulation with a quadrotor) and homogeneous teams (two mobile manipulators). Tasks combine ground-level manipulation and aerial inspection, requiring allocation across robots with complementary capabilities.

2) Evaluation Metrics: We evaluate HEART using metrics that capture both efficiency and planning quality:

- *#subtasks*: Average number of subtasks per instruction
- *Rounds*: Number of allocation cycles under budget b
- *Avg. tokens per subtask*: Average number of tokens used per subtask
- *Avg. time per subtask*: Average latency per subtask
- *Subtask SR*: Success rate (SR) of subtasks
- *Plan SR*: Success rate of task instructions that generate a plan that is both valid and physically executable

We also report *Allocation+Reasoning Token Count* and *Total Token Count* to quantify allocator overhead and overall token consumption, respectively.

3) Default Settings: Unless otherwise noted, the experiments are done with the following default settings. HEART is implemented using LangGraph, with task allocation relying on Sentence-BERT embeddings and tiktoken for token estimation. We use an LLM-as-Planner with chain-of-thought prompting as the default planner across all experiments. We also evaluate HEART with an additional planner, DELTA, a hybrid LLM–PDDL planning system with symbolic validation, to test compatibility with both natural language and symbolic planning pipelines.

Each scene provides five task instructions (15 total), and each instruction is repeated for 10 random seeds, yielding 150 trials per condition. Experiments use a per-round token budget of 20k. Subtasks have a retry limit of five; exceeding it sets `FAIL`. Results are reported as averages over seeds with 95% confidence intervals.

4) LLM Configuration: The decomposer agent uses reasoning model `o4-mini`, while all expert agents (capability, environment, path, feasibility, and constraint) use `gpt-4o`. The LLM-as-planner also uses `gpt-4o` with chain-of-thought. All models use temperature 0.0.

5) Hardware: Experiments ran on Ubuntu 22.04 with AMD 7900X, 32GB RAM, RTX 4080 SUPER; LLM calls use the OpenAI API.

B. Heterogeneous vs. Homogeneous Agents

We compare role-aligned heterogeneous agents with a homogeneous baseline. Both settings use the same structured prompt format, but the information provided differs: heterogeneous agents receive role-specific data, whereas the homogeneous baseline receives the full scene and robot data for every subtask. This baseline can in principle reason about all aspects of the task, but without specialization or filtering.

We evaluate this comparison using five metrics: Rounds, Avg. Token Count/subtask, Avg. Time/subtask, Subtask SR, and Plan SR. Results in Table II show that heterogeneous agents consistently require fewer rounds, use fewer tokens, and exhibit lower latency per subtask. This difference arises because homogeneous agents must handle larger inputs per query: under the 20k budget they can process fewer subtasks per round, which increases the total number of rounds, inflates token usage, and prolongs latency. Subtask SR is sometimes slightly lower with heterogeneous agents due to the lower amount of data owing to filtering, but plan success improve substantially across all scenes. For example, in

TABLE III: Allocator efficiency under different token budgets (Beechwood & Benevolence; 50 trials per allocator). Metrics for Stages 2 and 3 (Allocation+Reasoning): Subtask SR, Rounds, and Tokens (mean $\pm$ 95% CI).

Budget	Allocator	Subtask SR(%)	Rounds	Tokens (k)
10k	TypeBased	87.4	11.2 ± 1.5	122.5 ± 12.4
	LLM	95.7	6.6 ± 1.2	323.3 ± 41.9
	HEART	97.0	7.9 ± 1.3	102.6 ± 9.3
20k	TypeBased	87.8	6.3 ± 0.6	115.0 ± 12.7
	LLM	95.2	4.9 ± 0.5	224.6 ± 29.4
	HEART	97.6	5.2 ± 0.4	102.0 ± 8.7
40k	TypeBased	88.5	4.8 ± 0.3	120.3 ± 12.6
	LLM	95.2	4.0 ± 0.4	207.5 ± 23.6
	HEART	96.1	4.0 ± 0.3	99.3 ± 9.0

Benevolence, heterogeneous agents achieve a 24% higher plan SR despite a 1% drop in subtask SR. In Merom, heterogeneous agents reduce rounds by more than half while lowering time per subtask by 41% and increasing plan success by 18%. These results highlight that role-aligned specialization prioritizes the information most relevant to planning, leading to more efficient use of resources and higher-quality executable plans.

C. Allocator Comparison

We evaluate three allocation strategies under communication constraints (10k/20k/40k budgets; 50 trials per allocator per budget): (i) TypeBased, a rule-based baseline that deterministically maps each subtask type to a predefined agent, (ii) LLM allocator, where a general-purpose LLM directly decides agent assignments, and (iii) HEART, our subtask allocator with history-sensitive penalties and token budgeting. As described in Sec. IV, we define eleven subtask types and five expert roles; the fixed mapping used by TypeBased is provided in Table I. Results are summarized in Table III, with metrics measured on Stages 2 and 3 (Allocation+Reasoning): Subtask SR, Rounds, and Total Token Count.

At the tightest budget (10k), all allocators require more rounds, but HEART scales more gracefully: its rounds increase moderately compared to TypeBased baseline, which shows the largest growth. The LLM allocator achieves slightly fewer rounds overall, but this comes at the cost of two to three times higher token usage, reflecting substantial overhead from generating allocation decisions with LLMs. Across all budgets, HEART maintains the highest subtask SR (96–98%) while keeping token usage close to 100k regardless of budget. By contrast, TypeBased consumes fewer tokens than LLM but suffers from lower SRs ($< 89\%$) and higher rounds under tight budgets. These results indicate that HEART provides the most robust balance: it avoids the excessive overhead of LLM allocation and the rigidity of predefined rules, achieving high success with stable scaling under communication constraints.

D. Task Planning with Different Planners

We compare two planners on plan success rate and token count, using the same task instructions and the same per-round budget, each tested with and without HEART (Table IV). The planners are an LLM-based planner that generates an action sequence through chain-of-thought reasoning (LLM-CoT) and DELTA, an LLM-to-PDDL translation framework that constructs PDDL domain and problem files and solves them with a symbolic planner. LLM-CoT alone uses few tokens but often produces invalid or infeasible plans, with Plan SR of 20% in Benevolence and 0% in Merom. DELTA reaches 80% in Beechwood but drops to 20% in Benevolence and 10% in Merom.[2]

TABLE IV: Planning results for two planners, LLM-as-planner (LLM-CoT) and LLM-as-translator to a symbolic solver (DELTA), with and without HEART. Metrics: Plan SR and token count (mean $\pm$ 95% CI).

Scene	Planner	Plan SR (%)	Tokens (k)
Beechwood	LLM-CoT	52.0	12.5 ± 0.0
	HEART + LLM-CoT	82.0	130.2 ± 8.4
	DELTA	80.0	12.2 ± 0.2
	HEART + DELTA	96.0	136.2 ± 9.0
Benevolence	LLM-CoT	20.0	10.0 ± 0.0
	HEART + LLM-CoT	74.0	113.7 ± 9.0
	DELTA	20.0	11.4 ± 0.2
	HEART +DELTA	82.0	108.5 ± 6.1
Merom	LLM-CoT	0.0	13.0 ± 0.0
	HEART + LLM-CoT	63.3	150.4 ± 12.2
	DELTA	10.0	14.0 ± 0.2
	HEART +DELTA	86.7	139.6 ± 10.1

By contrast, adding HEART consistently improves outcomes: when paired with LLM-CoT it raises plan success to 82% in Beechwood, 74% in Benevolence, and 63.3% in Merom, while combined with DELTA it yields the highest success across all scenes (96%, 82%, and 86.7%). The performance gap is particularly highlighted in Benevolence and Merom, where many task instructions require reasoning about object properties such as size, weight, and accessibility. These feasibility checks often cause single-model baselines to fail, but the feasibility agent in HEART validates them and prevents infeasible plans.

Although HEART increases token usage (e.g., from 10–13k to 110–150k), this overhead translates into substantially higher plan validity and executability. In practice, data pruning can further reduce this overhead without changing the allocation or reasoning framework. Importantly, HEART is agnostic to the choice of planner and improves both settings: LLM-as-planner (LLM-CoT) and LLM-as-translator to a symbolic solver (DELTA), by providing validated context and constraints during synthesis.

VI. Conclusion

We introduced HEART, a heterogeneous multi-agent system for robotic task planning. HEART decomposes natural language instructions into reasoning subtasks, allocates them to role-aligned expert agents under a per-round token budgets, and integrates agent responses into plan synthesis. Across three household environments, HEART improves efficiency and plan success over homogeneous agents, baseline allocators, and single-planner baselines. It also strengthens

[2]Merom results are over 30 trials (3 instructions $\times$ 10 seeds). DELTA, built on Fast Downward, cannot model concurrency and therefore struggles with homogeneous multi-robot settings.

both LLM-as-planner (LLM-CoT) and LLM-as-translator to a symbolic solver (DELTA) by supplying validated context and constraints during synthesis. These results highlight the value of role specialization and allocation under per-round token budget for producing valid and executable task plans. HEART achieves robust and executable task plans in various settings, though at the cost of substantially higher token usage. This overhead can be mitigated through pruning and data reduction techniques, as shown in our experiments. Future work will extend the heterogeneity of HEART from the prompt level to the model level, where each expert role is paired with its own small or fine-tuned language model. This specialization would allow each agent to reason more efficiently within its domain and further reduce token usage while preserving collaborative reasoning.

REFERENCES

[1] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "Sayplan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *Proceedings of the Conference on Robot Learning*, vol. 229, 2023, pp. 23–72.

[2] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, "Delta: Decomposed efficient long-term robot task planning using large language models," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 10 995–11 001.

[3] D. Driess *et al.*, "PaLM-E: An embodied multimodal language model," in *Proceedings of the International Conference on Machine Learning*, vol. 202, 2023, pp. 8469–8488.

[4] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, "Coherent: Collaboration of heterogeneous multi-robot system with large language models," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 10 208–10 214.

[5] J. Hoffmann and B. Nebel, "The FF planning system: Fast plan generation through heuristic search," *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.

[6] A. Coles, A. Coles, M. Fox, and D. Long, "Forward-chaining partial-order planning," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 20, 2010, pp. 42–49.

[7] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[8] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024, pp. 286–299.

[9] T. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2024, pp. 8048–8057.

[10] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024, pp. 4311–4317.

[11] H. Chen, W. Ji, L. Xu, and S. Zhao, "Multi-agent consensus seeking via large language models," *arXiv preprint arXiv:2310.20151*, 2023.

[12] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," in *Proceedings of the International Conference on Learning Representations*, 2025.

[13] A. Singh *et al.*, "Improving test-time search for LLMs with backtracking against in-context value verifiers," in *ICLR Workshop on Reasoning and Planning for Large Language Models*, 2025.

[14] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-agent collaboration mechanisms: A survey of llms," *arXiv preprint arXiv:2501.06322*, 2025.

[15] M. Ahn *et al.*, "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," in *Proceedings of the Conference on Robot Learning*, vol. 205, 2023, pp. 287–318.

[16] S. S. Kannan, V. L. Venkatesh, and B.-C. Min, "SMART-LLM: Smart multi-agent robot task planning using large language models," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 12 140–12 147.

[17] K. Valmeekam, A. Olmo, S. Sreedharan, and S. Kambhampati, "Large language models still can't plan (a benchmark for llms on planning and reasoning about change)," in *NeurIPS Workshop on Foundation Models for Decision Making*, 2022.

[18] V. Pallagani *et al.*, "On the prospects of incorporating large language models (LLMs) in automated planning and scheduling (APS)," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 34, 2024, pp. 432–444.

[19] M. Fox and D. Long, "PDDL2.1: An extension to PDDL for expressing temporal planning domains," *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003.

[20] Y. Wu *et al.*, "SELP: Generating safe and efficient task plans for robot agents with large language models," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 2599–2605.

[21] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, "AutoTAMP: Autoregressive task and motion planning with llms as translators and checkers," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024, pp. 6695–6702.

[22] W. Chen, S. Koenig, and B. Dilkina, "Why solving multi-agent path finding with large language model has not succeeded yet," *arXiv preprint arXiv:2401.03630*, 2024.

[23] P. Li, V. Menon, B. Gudiguntla, D. Ting, and L. Zhou, "Challenges faced by large language models in solving multi-agent flocking," *arXiv preprint arXiv:2404.04752*, 2024.

[24] H. Babu, P. Schillinger, and T. Asfour, "Adaptive domain modeling with language models: A multi-agent approach to task planning," *arXiv preprint arXiv:2506.19592*, 2025.

[25] H. Cao, R. Ma, Y. Zhai, and J. Shen, "LLm-Collab: a framework for enhancing task planning via chain-of-thought and multi-agent collaboration," *Applied Computing and Intelligence*, vol. 4, no. 2, pp. 328–348, 2024.

[26] J. Chen, Z. Yang, H. G. Xu, D. Zhang, and G. Mylonas, "Multi-agent systems for robotic autonomy with llms," in *CVPR Workshop on Multi-Agent Embodied Intelligent Systems Meet Generative-AI Era: Opportunities, Challenges and Futures*, 2025.

[27] Z. Jia, H. Gao, F. Li, J. Liu, H. Li, and Q. Liu, "Triple-S: A collaborative multi-llm framework for solving long-horizon implicative tasks in robotics," *arXiv preprint arXiv:2508.07421*, 2025.

[28] Z. Lin, C. Tang, H. Ye, and H. Zhang, "Flowplan: Zero-shot task planning with llm flow engineering for robotic instruction following," *arXiv preprint arXiv:2503.02698*, 2025.

[29] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[30] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, "Market-based multirobot coordination: A survey and analysis," *Proceedings of the IEEE*, vol. 94, no. 7, pp. 1257–1270, 2006.

[31] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1495–1512, 2013.

[32] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE transactions on parallel and distributed systems*, vol. 13, no. 3, pp. 260–274, 2002.

[33] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling llm test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[34] I. Armeni *et al.*, "3D Scene Graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 5663–5672.

[35] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese bert-networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.

[36] P. Belcak *et al.*, "Small language models are the future of agentic ai," *arXiv preprint arXiv:2506.02153*, 2025.

[37] M. Yue, "A survey of large language model agents for question answering," *arXiv preprint arXiv:2503.19213*, 2025.

[38] C. Li *et al.*, "iGibson 2.0: Object-centric simulation for robot learning of everyday household tasks," in *Proceedings of the Conference on Robot Learning*, 2021.